SOLUTION OVERVIEW FOR CUSTOMERS

AISDLC

AI-Driven Software Development Lifecycle — From Business Requirement to Production with Governed AI Agents

Version	1.0
Generated	2026-11-03
Repository	Deepankar-AICloud/AISDLC-V1
Source commit	3e2b754 (main, 2026-11-03)
Classification	Customer — external

This overview describes the AISDLC platform and its reference implementation (the SmartServe IT Service & Access Request Portal). Facts and figures come from the repository and its GitHub history. No credentials, tenant identifiers or customer data are included.

Document Control

Table 0.1 — Document control

Document title	AISDLC — Solution Overview
Document ID	AISDLC-OVW-001
Version / status	1.0 — Issued for customer review
Audience	Business sponsors, IT leadership, product owners, architects, engineering and security leads
Purpose	Explain what AISDLC is, why it is needed, how it works, how it is governed, what it delivers, and how to adopt it
Companion document	SmartServe — Complete Application Solution Document (detailed technical reference)

Revision History

Table 0.2 — Revision history

Version	Date	Author	Description
1.0	2026-11-03	AISDLC team	First customer issue.

Table of Contents

1 Executive Summary
- 1.1 Why it matters
- 1.2 Proven on a real application
- 1.3 What we ask of you

2 What is AISDLC?

3 Why AISDLC?
- 3.1 The problem with traditional delivery
- 3.2 What AISDLC changes
- 3.3 Who benefits

4 Guiding Principles

5 How AISDLC Works
- 5.1 The lifecycle
- 5.2 Inside one stage
- 5.3 End-to-end interaction
- 5.4 How people interact with AISDLC

6 The AI Agent Team

7 Quality and Governance
- 7.1 Layers of control
- 7.2 Quality gates
- 7.3 When quality is not met
- 7.4 Human decision points

8 Security, Safety and Responsible AI

9 Traceability and Transparency

10 Architecture and Technology

11 Case Study — SmartServe

12 Benefits and Value

13 Adopting AISDLC
- 13.1 Prerequisites
- 13.2 Onboarding approach
- 13.3 Roles and responsibilities
- 13.4 Writing a good requirement issue

14 Current Capabilities, Limitations and Roadmap
- 14.1 Capabilities today
- 14.2 Known limitations
- 14.3 Roadmap

15 Frequently Asked Questions

16 Glossary

1 Executive Summary

AISDLC (AI-Driven Software Development Lifecycle) turns a business requirement into reviewed requirements, design, a delivery plan and working, tested code. It uses a team of specialised AI agents, each governed by automated quality controls and **human approval at every stage**. It runs entirely inside your GitHub repository, using GitHub Issues, Pull Requests, Actions and GitHub Copilot, and deploys to your cloud platform (Azure in the reference implementation).

AISDLC in one paragraph. A business user describes a need in a GitHub issue. AISDLC first checks that the description is complete enough to work with. If it is not, it asks for the missing details instead of guessing. It then drafts each standard delivery artifact in turn: Business Requirements (BRD), Product Requirements (PRD), Epics and User Stories, Architecture, Non-Functional Requirements (NFRs), a Task Plan, and finally Code with tests. Every draft is scored by an independent AI reviewer against configurable quality thresholds, revised if needed, and then submitted as a pull request for a human to approve. Only after approval does the next stage start. Everything is traceable: each requirement has a stable ID that can be followed from the original request to the code and the release.

1.1 Why it matters

Table 1.1 — Value summary

Business need	How AISDLC responds
Faster path from idea to working software	Agents draft each artifact in minutes. People spend their time reviewing and deciding, not writing boilerplate
Consistent, complete documentation	Every artifact follows a defined contract (required sections, ID formats, traceability links)
Control and accountability	Humans approve every stage; production release always needs explicit approval
Compliance and auditability	Stable requirement IDs, PR history, decision logs and quality scores form an audit trail
Safe use of AI	AI output is checked by a second model, by deterministic rules, and by people; requirement text is treated as data, never as instructions

1.2 Proven on a real application

AISDLC was used to deliver **SmartServe**, an Enterprise IT Service & Access Request Portal (.NET 8 API, React, Azure SQL, Microsoft Entra ID, Azure App Service). From a single GitHub issue it produced:

- Business requirements, product requirements, epics and stories.
- Architecture, NFRs and a 16-task delivery plan.
- A 55-file implementation with tests and infrastructure code.

All seven stages were approved through pull requests #24 to #34. Chapter 11 has the details.

1.3 What we ask of you

1. Provide a GitHub repository with Actions and GitHub Copilot access, and a cloud subscription for deployment.
2. Nominate reviewers for each stage: business (BRD/PRD), architecture, engineering and security.
3. Write requirements as GitHub issues, answer the platform's clarification questions, and review the stage pull requests.

2 What is AISDLC?

AISDLC is a **governed, AI-assisted delivery pipeline** that lives in your source repository. It is made of four parts.

Table 2.1 — AISDLC building blocks

Part	What it does	Where it lives
Orchestrator	Owns the workflow state. Decides which stage runs next, runs one stage per execution, and never skips a gate	Python control plane in the repository
AI agents	Specialists that each produce one artifact type (BRD, PRD, Epics, Architecture, NFR, Tasks, Code)	Agent definitions (instructions) in the repository; models served by GitHub Copilot
Quality control and model router	Scores every artifact independently; picks the right AI model for each task and explains why	Configuration files plus control-plane code
GitHub workflows	Run each stage, open pull requests, react to approvals and comments, and deploy	GitHub Actions

AISDLC **does not replace** your people, tools or governance. It is designed so that:

- the **orchestrator owns state**;
- **agents** only do specialised work;
- **quality control** evaluates;
- **humans** own every approval;
- the **model router** owns model selection;
- **GitHub Actions** runs everything.

Agents never call each other directly. Every hand-off goes through the orchestrator and a human-approved pull request.

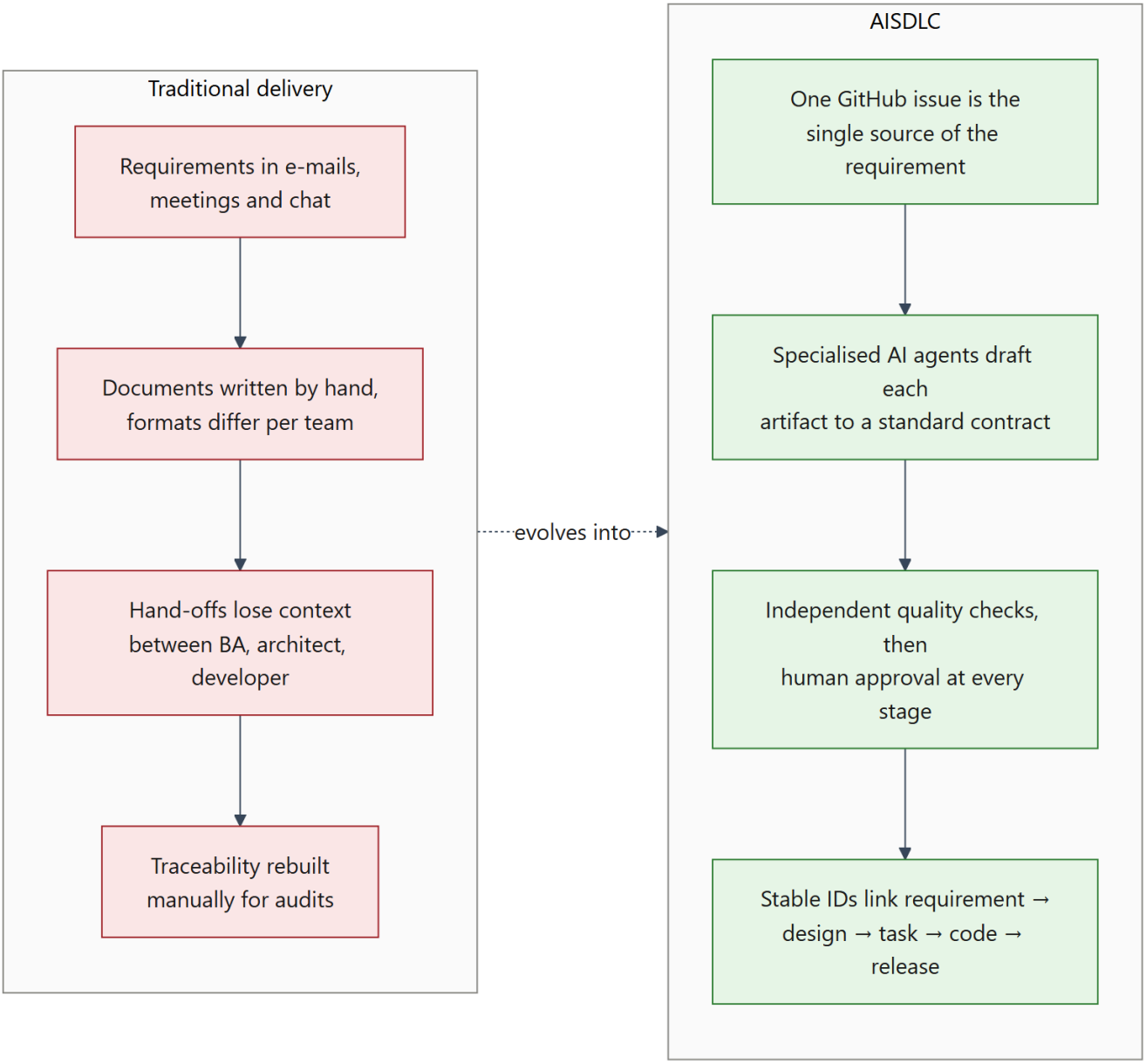


Figure 1 — From fragmented manual delivery to a governed AI-assisted lifecycle

3 Why AISDLC?

3.1 The problem with traditional delivery

- **Requirements are scattered** across e-mail, meetings and chat. Assumptions creep in and are rarely recorded.
- **Documentation is slow and inconsistent.** Business analysts, architects and engineers each write in their own format, and documents drift out of date.
- **Hand-offs lose context.** Each role re-interprets the previous one's output; design intent does not reach the code.
- **Traceability is rebuilt by hand** for audits and change requests, so the impact of a change is hard to see.
- **Quality checks happen late.** Gaps are found in testing or production, where they are most expensive to fix.
- **Ad-hoc use of AI tools** speeds up individuals but adds risk: invented requirements, unreviewed code, no record of which model produced what.

3.2 What AISDLC changes

Table 3.1 — Challenges and capabilities

Challenge	AISDLC capability
Incomplete requirements	A **readiness gate** checks for the business problem, objective and stakeholders, and **asks** for anything missing; it does not invent it
Inconsistent artifacts	**Artifact contracts** define required sections, ID formats and upstream references for every stage
Context lost between roles	Each agent receives the **approved upstream artifacts** as its input, so design builds on agreed requirements
Late quality feedback	An **independent quality gate** after every stage, with configurable thresholds and automatic revision
Weak traceability	**Stable IDs** (`BRD-REQ` , `PRD-FR` , `US` , `ARCH-COMP` , `NFR` , `TASK` , …) link each item to its source
Uncontrolled AI use	**Human approval** at each stage, **model confirmation** for high-impact stages, and a **decision log** of model choices
Slow, risky releases	Deterministic CI, security scanning, staged deployment and a **mandatory production approval**

3.3 Who benefits

- **Business sponsors:** faster turnaround from idea to reviewable solution, with clear status at every stage.
- **Product owners and analysts:** structured first drafts to refine instead of blank pages; open questions surfaced early.
- **Architects:** consistent architecture documents that trace to requirements; "proposed" design choices are clearly labelled.
- **Engineering:** a dependency-ordered task plan and a first implementation with tests, built and tested by the pipeline before review.

- **Security and compliance:** an audit trail of decisions, reviewers, quality scores and releases, plus built-in security scanning.

4 Guiding Principles

AISDLC is built on principles that are enforced in configuration and code, not just written down.

Table 4.1 — Principles and enforcement

#	Principle	How it is enforced
1	**Human in the loop** before progression	Every stage ends in a pull request; the next stage starts only after approval
2	**Quality before advancing**	An artifact that scores below its threshold, or breaks its contract, cannot progress
3	**Never invent requirements**	The readiness gate pauses and asks; design choices that are not confirmed must be labelled *(proposed)*
4	**Full traceability**	Required ID prefixes and upstream references per stage; a traceability report per run
5	**Security by design**	Code scanning gate, secret scanning, least-privilege tokens, OIDC to the cloud, managed identities
6	**Configuration over hard-coding**	Agents, models, thresholds, contracts and environments are YAML configuration
7	**Honest telemetry**	Token usage and cost are recorded only when known; otherwise they are labelled *unavailable*, never estimated as fact
8	**Mandatory production approval**	Production release requires a named approver and an explicit confirmation
9	**Untrusted input is data, not instructions**	Issue and comment text is clearly separated from the agents' trusted instructions

5 How AISDLC Works

5.1 The lifecycle

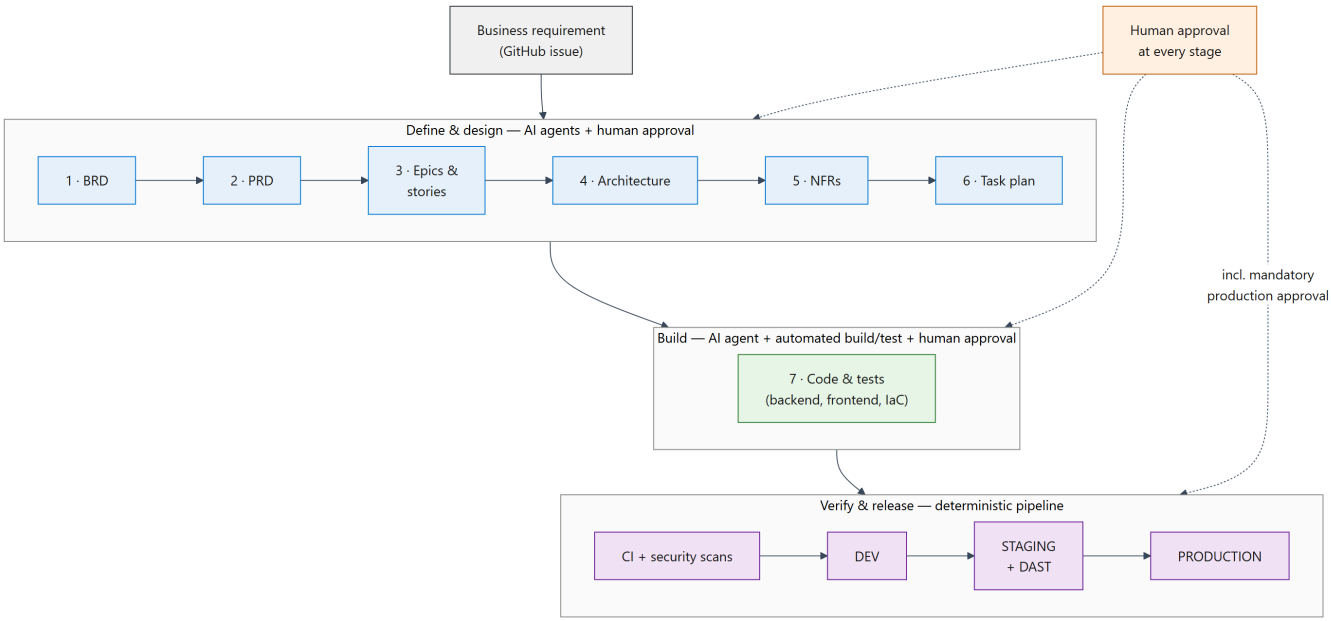


Figure 2 — The AISDLC lifecycle: define, design, build, verify and release

Table 5.1 — Stages, artifacts and reviewers

Stage	Input	Output (artifact)	Reviewer
0 · Intake	Business need written as a GitHub issue	Validated requirement context	Requester
1 · BRD	Issue and clarifications	Business Requirements Document (`BRD-REQ-nnn`)	Business owner
2 · PRD	Approved BRD	Product / functional specification (`PRD-FR` , `PRD-NFR`)	Product owner
3 · Epics and stories	Approved PRD	Epics, features, user stories with acceptance criteria	Product owner
4 · Architecture	Approved PRD	Architecture document with diagrams (`ARCH-COMP`)	Architect
5 · NFRs	Approved architecture	Verifiable non-functional requirements (`NFR-nnn`)	Architect / security
6 · Task plan	Architecture and NFRs	Dependency-ordered tasks with tests (`TASK` , `UT` , `FT`)	Engineering lead
7 · Code	Task plan and architecture	Source code, tests and infrastructure code, built and tested by the pipeline	Engineering + code owners
8 · Verify and release	Merged code	CI, security scans, DEV → STAGING (with DAST) → PRODUCTION	Release manager (mandatory)

5.2 Inside one stage

Every stage follows the same governed pattern:

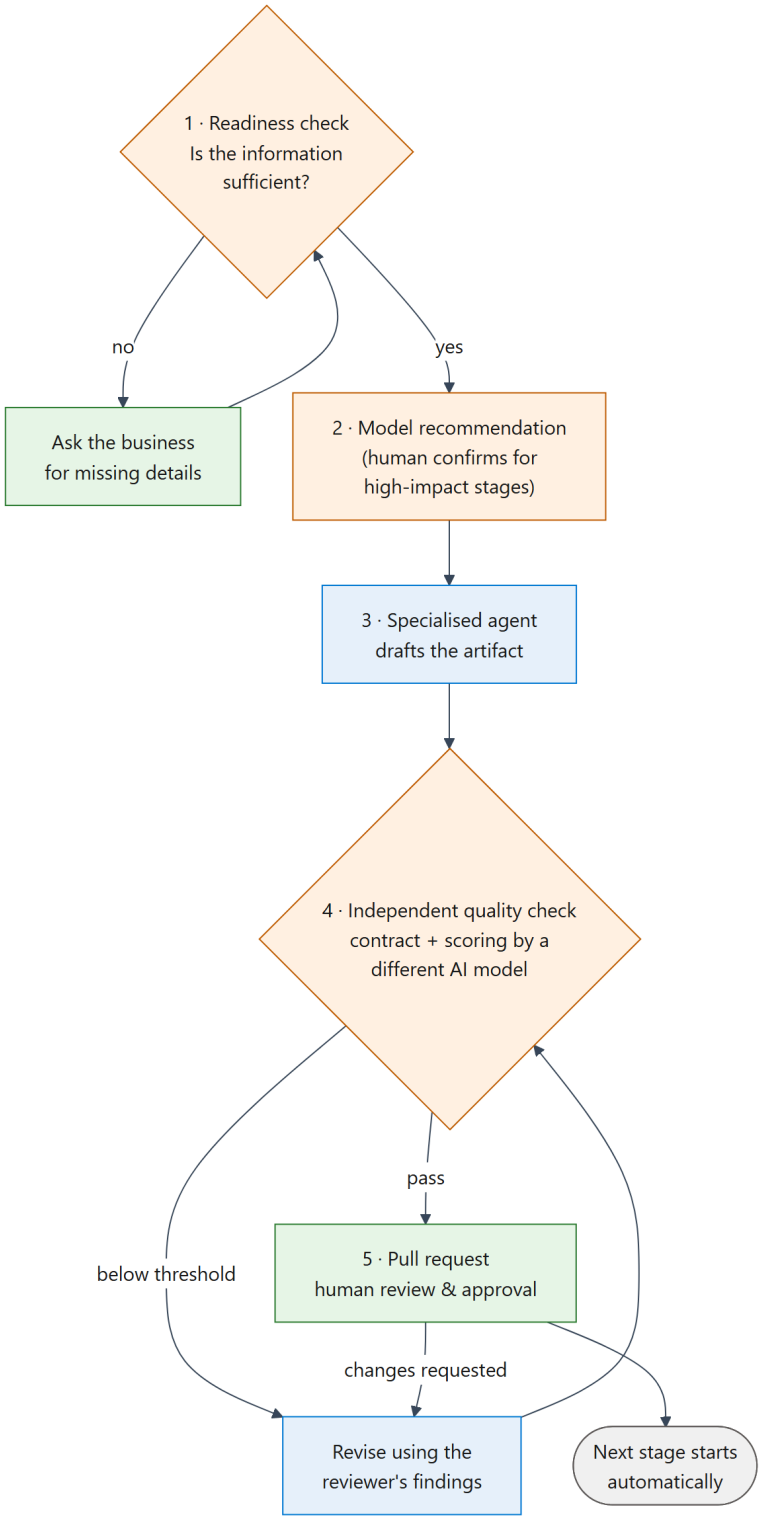


Figure 3 — Anatomy of one stage: readiness, model choice, generation, quality control, human approval

1. **Readiness check.** Is there enough information? If not, the issue is labelled *awaiting information* and the questions are posted. Answering in a comment resumes the stage automatically.

2. **Model recommendation.** The model router picks a model to match the task's complexity and explains why. For high-impact stages (BRD, PRD, Architecture, NFR, Code) a person confirms the choice with `/model approve`, or selects another model.
3. **Generation.** The specialised agent drafts the artifact against its contract, using the approved upstream artifacts.
4. **Quality control.** Deterministic checks (required sections, IDs, diagrams, build and tests for code) plus scoring by a *different* AI model. Below the threshold, the agent **revises its previous draft** using the reviewer's findings rather than starting over. After repeated failures the stage pauses and asks a person for guidance.
5. **Approval.** The artifact is committed to a pull request. A reviewer approves it, or comments to request changes, and the next stage starts automatically.

5.3 End-to-end interaction

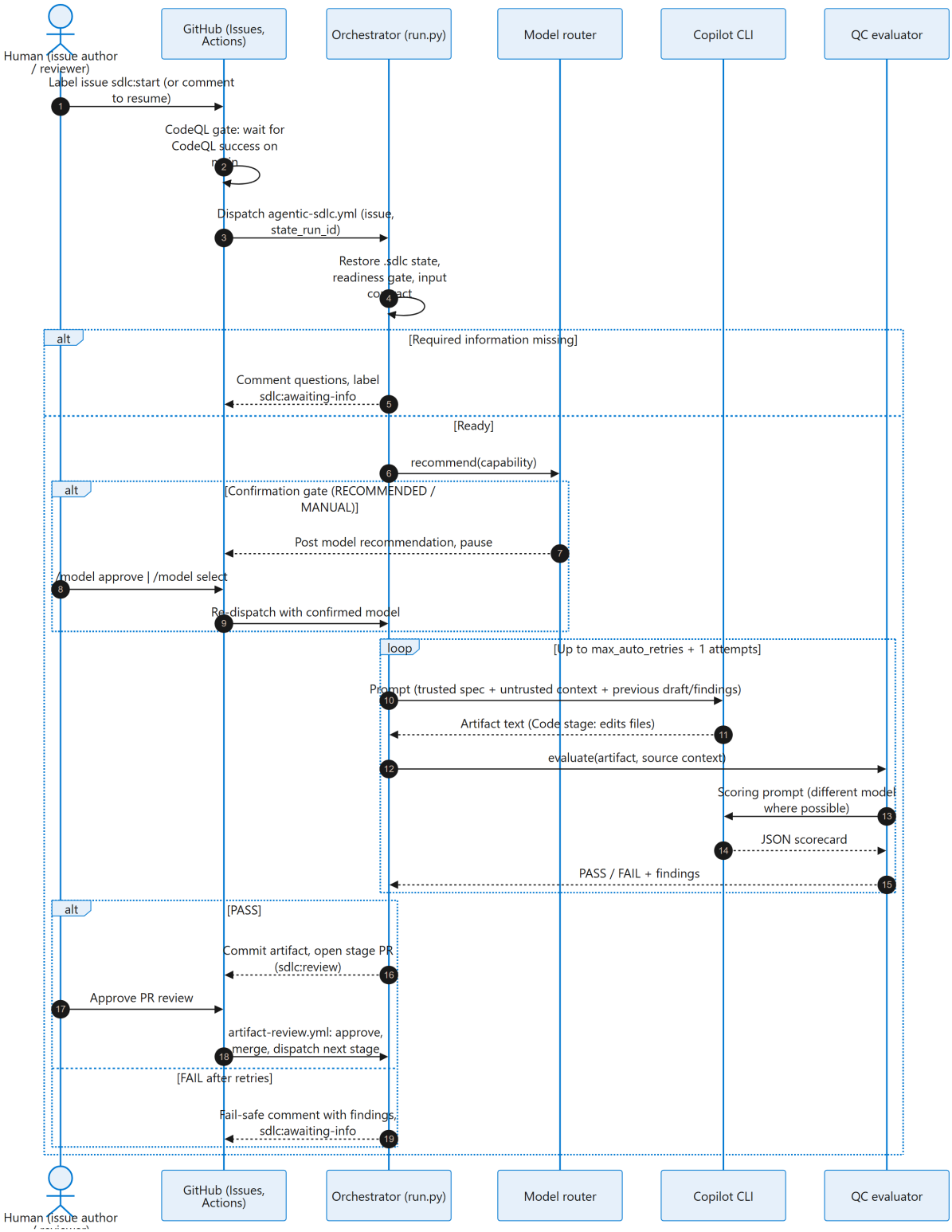


Figure 4 — Interaction between people, GitHub, the orchestrator, the AI models and quality control

5.4 How people interact with AISDLC

Table 5.2 — Operating the platform

Action	How

Action	How
Start a requirement	Create a GitHub issue describing the business problem, objective and stakeholders; add the label `sdlc:start`
Answer questions	Reply in the issue. The stage resumes automatically
Confirm or change the AI model	Comment `/model approve` or `/model select <model> <reason>`
Request a targeted change	Comment `/agent revise <notes>` on the stage pull request (also `regenerate`, `rerun-qc`, `explain`, `status`, `impact`)
Approve a stage	Approve the stage pull request
Release to production	Run the production workflow, type `promote`, and approve as the designated reviewer

6 The AI Agent Team

Table 6.1 — The AI agent team (thresholds are configurable per stage)

Agent	Role	Produces	Quality threshold	Human review
BRD Agent	Business analyst	Business problem, objectives, stakeholders, scope, requirements, acceptance criteria	90 / 100	Required
PRD Agent	Product manager	Functional and non-functional specification traced to the BRD	90 / 100	Required
Epic & Feature Agent	Product owner	Epics, features and user stories with acceptance criteria	85 / 100	Approval through PR
Architect Agent	Solution architect	Architecture document, Mermaid diagrams, components, security and deployment views	90 / 100	Required
NFR Agent	Quality engineer	Verifiable NFRs backed by the sources; open questions instead of invented targets	88 / 100	Approval through PR
Task Planning Agent	Engineering lead	Dependency-ordered tasks with unit and functional test tasks	85 / 100	Approval through PR
Engineer Agent	Software engineer	Source code, tests and infrastructure code; must build and pass tests in the pipeline	Build and tests must pass; score ≥ 60 (currently)	Required (code owners)
Quality Control	Independent reviewer	Scorecards, findings and recommendations for every artifact	—	—
Security, test and release functions	Scanners and pipelines	CodeQL, secret scanning, OWASP ZAP, CI tests, staged deployment, release evidence	Tool verdicts	Mandatory for production

Agent behaviour is defined in plain-language instruction files that your team can read, review and tailor. Changing an agent is a reviewed code change, not a hidden prompt.

7 Quality and Governance

7.1 Layers of control

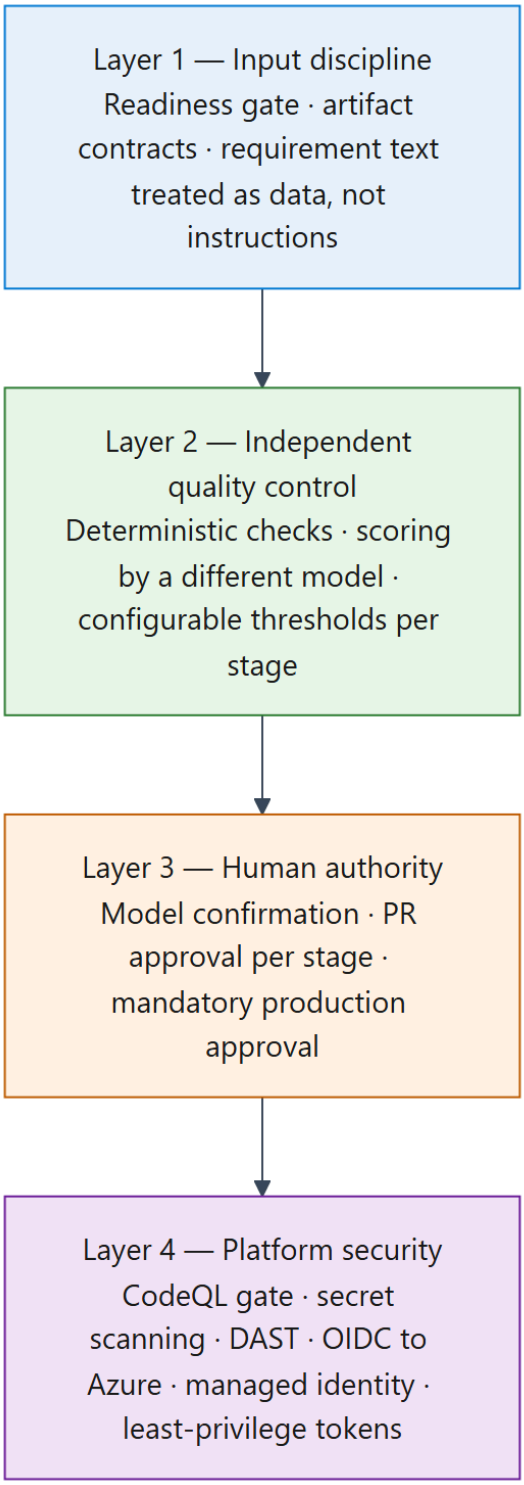


Figure 5 — Four layers of control around every AI-generated artifact

7.2 Quality gates

Every artifact is evaluated by quality control (QC), which is independent of the agent that produced it:

- **Deterministic checks.** Required sections, required ID prefixes, diagrams where mandated, and output-contract compliance. For code, the pipeline itself runs the builds and tests (`dotnet build` / `test`, `npm build`, `vitest`) and records the results as authoritative evidence.
- **Semantic scoring.** A *different* AI model from the one that generated the artifact scores it against stage-specific dimensions (for example completeness, clarity, traceability, testability, consistency, security, maintainability). It penalises content that is not supported by the source requirement.
- **Thresholds.** Each stage has a configurable pass mark (Table 6.1). Any critical issue — a missing section, a build failure, a contract violation — fails the stage regardless of score.

7.3 When quality is not met

QC does not assume that a stronger model is always the answer. It diagnoses the likely cause:

Table 7.1 — QC diagnosis and actions

Diagnosed cause	Action
Structural gaps (missing sections, IDs, traceability)	Revise with the reviewer's findings; then escalate to a stronger model, or to a person if the model was confirmed by a person
Possibly unsupported or invented content	Stop and **ask a person** to confirm the disputed requirements
Unclear cause	Retry with refined context, then escalate
Repeated failure	Pause the stage, post the findings on the issue, and wait for human guidance

Revisions **edit the previous draft** (for code, the previous attempt's changes are restored), so good content is kept and only the flagged items change.

7.4 Human decision points

Table 7.2 — Human decision points

Decision	Who	Mechanism
Is the requirement clear enough?	Requester / product owner	Answer the readiness questions in the issue
Which AI model to use for high-impact stages	Stage owner	`/model approve` or `/model select`
Is the artifact acceptable?	Stage reviewer	Pull-request approval or a change request
Can code be merged?	Code owners	Pull-request review (CODEOWNERS)
Can we go to production?	Release approver	Protected `production` environment and explicit confirmation

8 Security, Safety and Responsible AI

Table 8.1 — Security and responsible-AI controls

Area	Control
Prompt-injection resistance	Agent instructions are marked *trusted*; issue and comment text is marked *untrusted data, not instructions*; only allowlisted `/agent` and `/model` commands are recognised
No invented requirements	Readiness gate; QC penalises unsupported content; unconfirmed design choices must be labelled *(proposed)*
Credential protection	Repository tokens are removed from the AI process environment; the code agent runs without stored push credentials; deployments use OIDC federation (no stored cloud secrets)
Scope control for code	The engineer agent may change only approved folders (application source and infrastructure). Changes elsewhere, including workflow files, are reverted automatically
Code security	The CodeQL analysis must pass on the main branch before any AISDLC stage runs; secret scanning on pushes and pull requests; OWASP ZAP dynamic testing in staging
Separation of duties	The model that scores an artifact differs from the model that wrote it, where possible; humans approve every stage
Runtime security of delivered apps	Delivered solutions follow the approved architecture. In the reference app: Entra ID authentication, server-side role authorization, managed identity, Key Vault, TLS 1.2+
Auditability	Pull-request history, model decision records, QC scorecards, run reports and release evidence packs

9 Traceability and Transparency

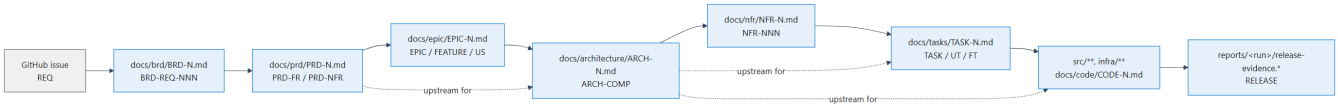


Figure 6 — Generated artifacts and the traceability chain

Traceability chain: `REQ → BRD-REQ → PRD-FR → EPIC → FEATURE → US → ARCH-COMP → TASK → CODE → UT → FT → SEC → RELEASE`.

Each run produces reports alongside the artifacts:

Table 9.1 — Transparency reports

Report	Content
Workflow report	Stage, agent, status, QC score and retries
Quality report	Scores per stage and dimension, with findings
Model decision report	Recommended vs selected model, who decided, and why
Usage and cost report	Model, latency, retries; token counts and cost where the provider reports them, otherwise labelled *unavailable*
Traceability report	IDs found per stage, plus gaps and orphans
Release evidence pack	Deployment and health-check results for production releases (retained 90 days)

> **Note on cost reporting.** AISDLC uses GitHub Copilot, whose command-line interface does not currently expose token counts. AISDLC therefore reports token usage and cost as *unavailable* rather than estimating them. Model choices, retries and latency are always recorded.

10 Architecture and Technology

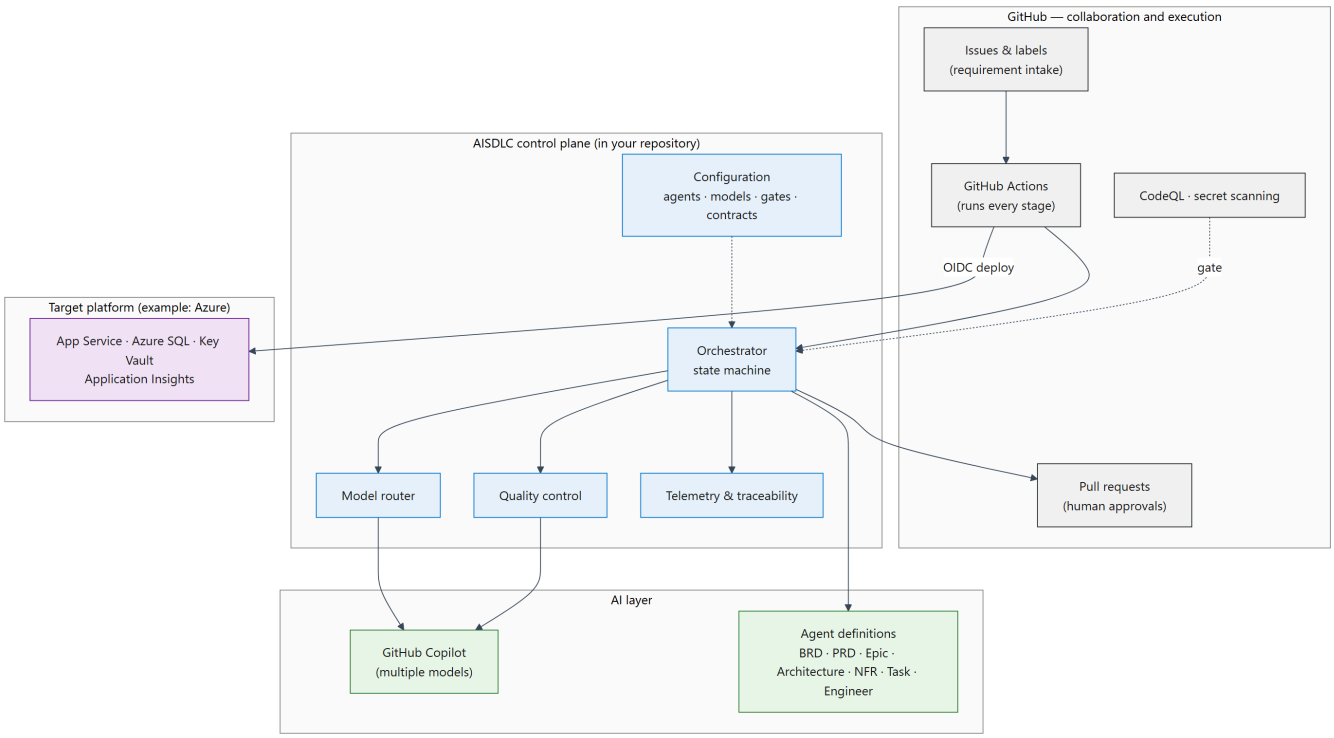


Figure 7 — AISDLC platform architecture

Table 10.1 — Technology stack

Layer	Technology
Collaboration and execution	GitHub Issues, Pull Requests, Actions, CODEOWNERS, environments
AI	GitHub Copilot (model families configured in tiers: fast, balanced, high-reasoning, with automatic fallback)
Control plane	Python 3.13 orchestrator, model router, quality control, readiness and contract gates, telemetry, traceability
Configuration	YAML: agents, models and routing, quality gates, artifact contracts, readiness rules, budgets, environments, impact rules
Code security	GitHub CodeQL, secret scanning (gitleaks), OWASP ZAP
Target platform (reference)	Azure App Service with deployment slots, Azure SQL, Key Vault, Application Insights, Bicep infrastructure as code, GitHub OIDC

Model tiers (configurable).

Tier	Typical use	Preferred / fallback
Fast	Classification, formatting, simple summaries	gpt-5-mini / claude-haiku-4.5
Balanced	BRD, PRD, epics, stories, test cases	gpt-5.5 / claude-sonnet-4.6
High reasoning	Architecture, complex code, security reasoning	claude-sonnet-5 / gpt-5.5

Model names are configuration, not code, and change as the provider's catalogue evolves.

Design choices that keep you in control:

- **No external AI platform to operate.** The control plane is code in your repository and runs on GitHub Actions.
- **Model-agnostic.** Swap or add models in configuration. When a model is not available to your account, AISDLC records the fallback it used.
- **Repository-native artifacts.** Every document is Markdown with Mermaid diagrams, version-controlled and reviewable like code.
- **Composable.** Stages can be enabled or disabled, their reviewers changed and thresholds tuned without code changes.

11 Case Study — SmartServe

Requirement. Employees and IT teams needed a secure, centralised way to submit, review and track IT service and access requests. The constraints were Microsoft Entra ID authentication, server-side authorization, Azure SQL via Entity Framework Core, and managed identity with Key Vault.

What AISDLC delivered from one GitHub issue (#23):

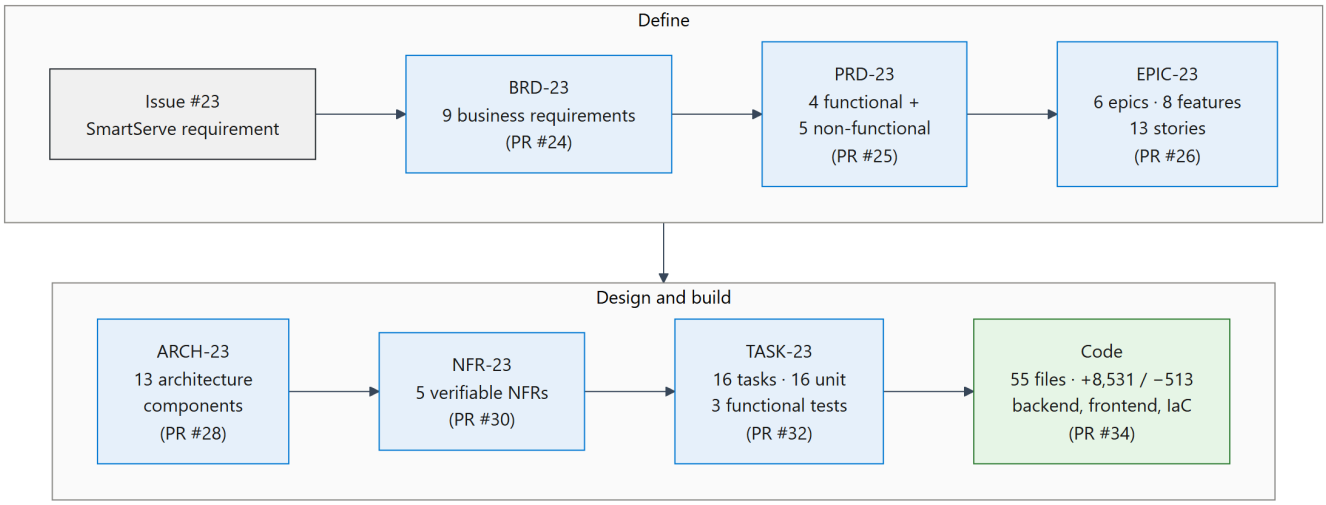


Figure 8 — Artifacts AISDLC produced for SmartServe, each approved through a pull request

Table 11.1 — SmartServe artifacts

Stage	Artifact	Content	Approved in
BRD	`docs/brd/BRD-23.md`	9 business requirements, stakeholders, scope, acceptance criteria	PR #24
PRD	`docs/prd/PRD-23.md`	4 functional + 5 non-functional requirements traced to the BRD	PR #25
Epics and stories	`docs/epic/EPIC-23.md`	6 epics, 8 features, 13 user stories	PR #26
Architecture	`docs/architecture/ARCH-23.md`	13 architecture components, diagrams; confirmed vs *(proposed)* choices labelled	PR #28
NFRs	`docs/nfr/NFR-23.md`	5 verifiable NFRs; performance, retention and DR recorded as open questions rather than invented	PR #30
Task plan	`docs/tasks/TASK-23.md`	16 tasks, 16 unit-test tasks, 3 functional-test tasks	PR #32
Code	`src/backend`, `src/frontend`, `infra`	55 files changed (+8,531 / −513 lines), built and tested by the pipeline	PR #34

What the case study demonstrated:

- **The platform asks instead of guessing.** When the intake lacked business context, it paused and requested the problem, objective and stakeholders. Later stages surfaced open decisions (notification triggers, role scoping, retention) as questions for the business.
- **Quality gates change the outcome.** Several stages needed more than one iteration. Reviewer findings were posted on the issue, product owners answered them in comments, and the agents revised their drafts until the gate passed.
- **People stayed in control.** Model choices were confirmed for high-impact stages, and every artifact was merged only after pull-request approval.
- **The output is consistent.** Every artifact follows its contract and carries traceable IDs back to the original requirement.

Next steps for SmartServe are the standard verification and release stages: CI, security scanning, DEV and STAGING deployment with DAST, and an approved production promotion. These are covered in the companion *SmartServe — Complete Application Solution Document*.

12 Benefits and Value

Table 12.1 — Benefits and suggested measures

Benefit	How it is achieved	How to measure it in your organisation
Shorter time from idea to reviewable design	Agents draft each artifact; people review	Elapsed time from issue creation to approved architecture or task plan
Higher documentation quality and consistency	Contracts, quality thresholds, automatic revision	QC scores per stage; number of review rounds
Fewer requirement gaps reaching development	Readiness gate, open-question handling, NFR discipline	Clarification questions raised before vs after development
End-to-end traceability for audit	Stable IDs and traceability reports	Coverage of requirement-to-test links; audit preparation effort
Controlled, explainable AI use	Model router, decision log, human approvals	Share of stages with human-confirmed models; override reasons
Safer releases	Security gates, staged deployment, mandatory approval	Failed releases; security findings caught pre-production

AISDLC records the data behind these measures (stage timestamps, QC scores, retries, model decisions) in its run reports. We recommend agreeing baseline values at the start of an engagement so that improvements can be measured objectively.

13 Adopting AISDLC

13.1 Prerequisites

Table 13.1 — Prerequisites

Area	Requirement
Source control	A GitHub repository (organisation or enterprise) with GitHub Actions enabled and permission for Actions to create pull requests
AI	GitHub Copilot access, and a token for the Copilot CLI stored as a repository secret
Security	GitHub CodeQL (default setup) and secret scanning enabled
Cloud (for deployment)	An Azure subscription, resource groups per environment, and an Entra ID application with a GitHub OIDC federated credential
Governance	GitHub environments (`dev` , `staging` , `production`) with required reviewers on production; CODEOWNERS teams
People	Named reviewers for business, product, architecture, engineering, security and release

13.2 Onboarding approach

Table 13.2 — Adoption phases (indicative; durations are agreed per engagement)

Phase	Activities	Outcome
1 · Set-up	Install the control plane and workflows in your repository; configure secrets, environments, CODEOWNERS and models; tailor agent instructions to your standards	Platform ready
2 · Pilot	Run one real requirement end to end with your reviewers; tune thresholds, contracts and reviewer assignments	Validated way of working; baseline metrics
3 · Scale	Roll out to more teams and requirement types; add organisation-specific agents or contracts (for example, compliance checklists)	Repeatable delivery model
4 · Optimise	Review QC and model-decision reports; adjust routing, thresholds and stage policies	Continuous improvement

13.3 Roles and responsibilities

Table 13.3 — RACI (R responsible, A accountable, C consulted, I informed)

Activity	Requester	Product owner	Architect	Engineering lead	Security	Release manager	AISDLC

Activity	Requester	Product owner	Architect	Engineering lead	Security	Release manager	AISDLC
Write the requirement issue	R/A	C					I
Answer readiness questions	R	A	C				I
Approve BRD / PRD / Epics	C	R/A	C	I	I		Drafts
Approve architecture / NFRs	I	C	R/A	C	C		Drafts
Approve task plan / code		I	C	R/A	C		Drafts, builds, tests
Approve production release	I	C	I	C	C	R/A	Evidence pack

13.4 Writing a good requirement issue

A good issue covers the following, in plain business language:

- The **business problem**.
- The **objective** or outcome.
- The **stakeholders and roles**.
- The **scope**.
- Any **constraints** (technology, security, compliance).

State what you know, and say explicitly what is still undecided. AISDLC will record undecided items as open questions rather than guessing.

14 Current Capabilities, Limitations and Roadmap

14.1 Capabilities today

Table 14.1 — Capability status

Capability	Status
Requirement intake with readiness gate and clarification loop	AVAILABLE
AI-generated BRD, PRD, epics and stories, architecture, NFRs and task plan, with quality gates and PR approval	AVAILABLE
AI-generated code with pipeline-run build and tests, scoped file changes, and revision across attempts	AVAILABLE
Model router with human confirmation, escalation, fallback and decision log	AVAILABLE
CI, secret scanning, CodeQL gate, staged deployment with DAST, mandatory production approval, release evidence	AVAILABLE
AI agents for code review, functional testing, performance and release	PLANNED — handled today by deterministic CI/CD tools and human review

14.2 Known limitations

- **AI output still needs expert review.** Quality gates reduce rework but do not replace reviewers. Clear, complete requirements are the biggest driver of fast, high-scoring stages.
- **Iterations vary by stage.** Design stages with many open business decisions may need several review rounds. Answering clarifications in one consolidated comment works best.
- **Token and cost visibility** depend on the AI provider. With the Copilot CLI, tokens and costs are reported as *unavailable*.
- **The code-stage pass mark is currently 60/100.** The build and tests are mandatory, and human code review remains the deciding control. The threshold is configurable and is expected to rise as the stage matures.

14.3 Roadmap

Table 14.2 — Roadmap (no dates implied)

Item	Benefit
Further hardening of comment-driven automation (input handling, member-only commands)	Stronger protection against malicious input
Fine-grained tool permissions for the engineer agent	Tighter least-privilege for AI-driven code changes
AI agents for code review and functional test generation as pipeline stages	More automation after the code stage, still human-approved
Per-stage dashboards (lead time, QC trends, review rounds)	Easier measurement of value
Provider-reported token usage and cost where available	Full cost transparency
Organisation templates (compliance checklists, architecture standards) as contracts	Built-in alignment with your standards

15 Frequently Asked Questions

Does AISDLC replace business analysts, architects or developers? No. It removes much of the drafting effort and enforces consistency. Every artifact is reviewed and approved by the responsible person, and accountability stays with your team.

Can the AI invent requirements? AISDLC is designed to prevent this:

- The readiness gate stops when information is missing.
- The reviewer model penalises unsupported content.
- Unconfirmed design choices must be labelled *(proposed)*.
- A person approves every artifact.

Which AI models are used, and can we choose? Models are configured in tiers and recommended per stage. For high-impact stages a person confirms the model or selects another, and every decision is logged with its reason.

Where do our code and data go? Artifacts and code stay in your GitHub repository. AI calls go through GitHub Copilot under your organisation's Copilot terms and policies. Deployments go to your own cloud subscription using OIDC, without stored cloud credentials.

Can we adapt it to our standards? Yes. Agent instructions, artifact contracts, quality thresholds, reviewers, environments and model routing are all configuration in your repository.

What if a stage keeps failing the quality gate? The stage pauses and posts the reviewer's findings on the issue. A short reply with the missing decisions resumes it, and the agent revises its previous draft rather than starting over.

Do we need Azure? The reference implementation deploys to Azure. The define, design and build stages are cloud-agnostic; the deployment workflows can be adapted to other targets.

16 Glossary

Table 16.1 — Glossary

Term	Meaning
AISDLC	AI-Driven Software Development Lifecycle
BRD / PRD	Business Requirements Document / Product Requirements Document
NFR	Non-functional requirement (security, performance, availability, …)
Artifact contract	Rules an artifact must satisfy: required sections, ID formats, upstream references
Quality gate (QC)	Independent evaluation with a pass mark that an artifact must meet before review
Readiness gate	Check that the requirement contains the information needed before generation
Model router	Component that recommends and selects AI models by task complexity
Orchestrator	Control-plane component that owns the workflow state and runs one stage at a time
Pull request (PR)	GitHub review mechanism used for human approval of every stage
CodeQL / DAST	Static code analysis / dynamic application security testing
OIDC	OpenID Connect, used to deploy to the cloud without stored secrets